Министерство образования и науки Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого

Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта
Направление: 02.03.01 «Математика и компьютерные науки»

Отчёт о выполнении лабораторных работ
По дисциплине: «Методы проектирования баз данных»

Выполнил студент
группы 5130201/40002 _____Семенов И. А.

Проверил
преподаватель _____Попов С. Г.

«____»_____ 2026г.

Санкт-Петербург
2026

Содержание

Введение **2**

1 Постановка задачи **3**

2 Основная часть **4**

2.1 Лабораторная работа № 1 4

2.1.1 Задача 4

2.1.2 Создание представления 4

2.1.3 Проверка представления 5

2.1.4 Запрос с условием 7

2.1.5 Охват форматов по категориям 8

2.2 Лабораторная работа № 2 10

2.2.1 Задача 10

2.2.2 Реализация триггеров 10

2.2.3 Проверка работы триггеров 14

2.2.4 Схема базы данных 16

2.3 Лабораторная работа № 3 18

2.3.1 Задача 18

2.3.2 Создание пользователей и назначение прав 18

2.3.3 Тестовые примеры 19

2.3.4 Результаты тестирования 21

2.4 Лабораторная работа № 4 23

2.4.1 Задача 23

2.4.2 Сценарий эксперимента 23

2.4.3 Результат 26

3 Заключение **27**

Введение

1 Постановка задачи

2 Основная часть

2.1 Лабораторная работа № 1

2.1.1 Задача

В ходе лабораторной работы необходимо:

1. создать представление, содержащее результат группировки данных с двумя агрегатными функциями `COUNT`;
2. составить простой запрос к созданному представлению с условием `WHERE`;
3. составить запрос, в котором представление используется совместно с операциями соединения `JOIN`.

2.1.2 Создание представления

Необходимо создать представление, которое для каждого начертания хранит число поддерживаемых форматов и количество градаций насыщенности.

Одно начертание может храниться в таблице `Typeface` в нескольких форматах и градациях насыщенности. Чтобы не выполнять одинаковый подсчёт в каждом запросе, создаётся отдельное представление.

В нём для каждого начертания хранятся название, идентификатор гарнитуры, число форматов и число градаций насыщенности. Определение представления приведено в листинге 1.

В формулах используются обозначения: T — таблица `Typeface`, F — таблица `Format`, W — таблица `Weight`, FF — таблица `Font_family`, C — таблица `Category`, V — созданное представление. Символ $\bowtie_k$ обозначает соединение по полю k, σ — выборку, π — проекцию, γ — группировку с агрегированием.

Листинг 1: Создание представления

```
CREATE OR REPLACE VIEW typeface_format_weight_stats_view AS
SELECT
    t.id_font_family,
    t.name AS typeface_name,
    COUNT(DISTINCT f.id_format) AS format_count,
    COUNT(DISTINCT w.id_weight) AS weight_count
FROM typeface AS t
JOIN format AS f ON f.id_format = t.id_format
JOIN weight AS w ON w.id_weight = t.id_weight
GROUP BY
    t.id_font_family,
    t.name;
```

Агрегатная функция `COUNT(DISTINCT ...)` подсчитывает количество различных значений. Поэтому повторяющиеся форматы и градации насыщенности учитываются только один раз. Для получения фактических форматов и градаций насыщенности таблица `Typeface` соединяется с таблицами `Format` и `Weight`.
Создание представления описывается формулой

$$V = \gamma_{\substack{id_font_family, name;\\ \text{count_distinct}(id_format)\to format_count,\\ \text{count_distinct}(id_weight)\to weight_count}} \left(\left(T \bowtie_{id_format} F \right) \bowtie_{id_weight} W \right).$$

2.1.3 Проверка представления

Текст запроса: Вывести первые десять начертаний из созданного представления вместе с числом поддерживаемых форматов и градаций насыщенности.
Запрос приведён в листинге 2.

Листинг 2: Проверка содержимого представления

```
SELECT
        v.id_font_family,
        v.typeface_name,
        v.format_count,
        v.weight_count
FROM typeface_format_weight_stats_view AS v
ORDER BY v.id_font_family, v.typeface_name
LIMIT 10;
```

Реляционная формула имеет вид

$$R_1 = \pi_{\substack{id_font_family, typeface_name,\\ format_count, weight_count}}(V).$$

План и результат выполнения показаны на рисунках 1 и 2.

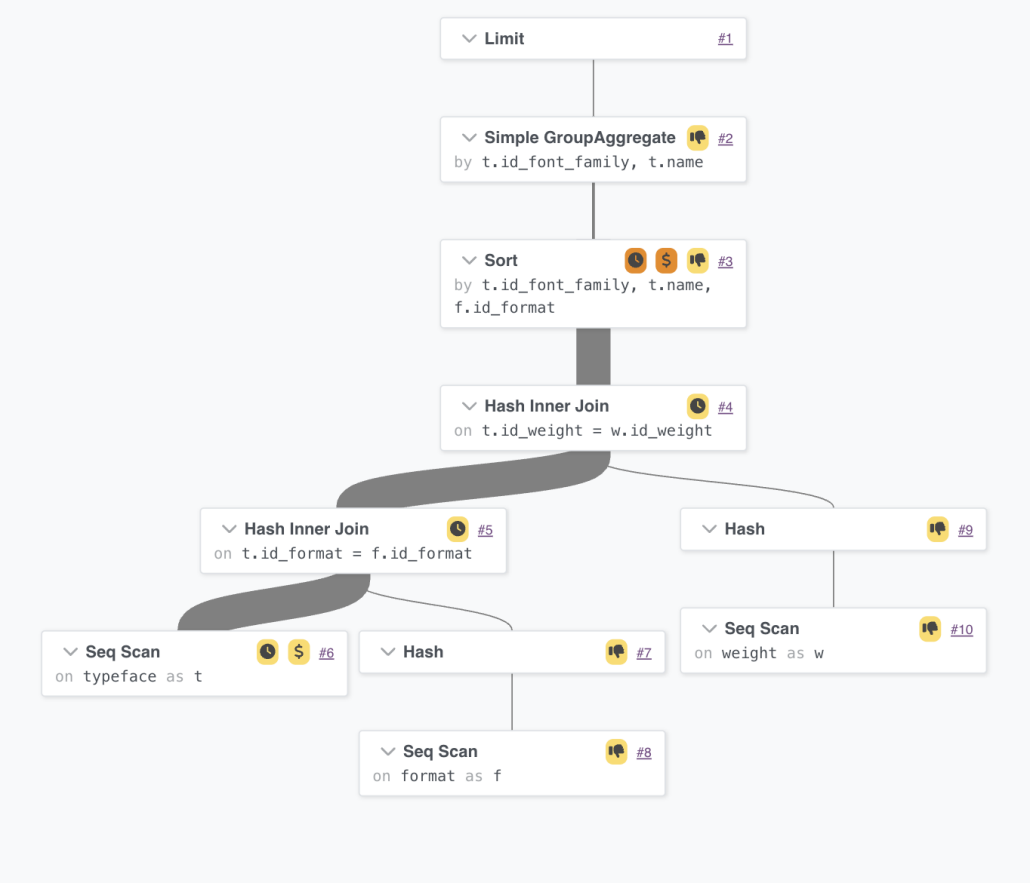


Рис. 1: План выполнения запроса 1

PostgreSQL читает таблицы `Typeface`, `Format` и `Weight`, затем соединяет их по полям `id_format` и `id_weight`. Полученные строки сортируются по полям группировки, после чего узел `GroupAggregate` рассчитывает количество различных форматов и градаций насыщенности каждого начертания.

```
cw-sem4 — zsh    ✓ PostgreSQL: run current f…
/dev/fd/12:25: command not found: compdef
 id_font_family | typeface_name | format_count | weight_count
----------------+---------------+--------------+--------------
              1 | Typeface 1    |            1 |            5
              2 | Typeface 2    |            1 |            2
              3 | Typeface 3    |            1 |            9
              4 | Typeface 4    |            1 |            7
              5 | Typeface 5    |            1 |            5
              6 | Typeface 6    |            1 |            8
              7 | Typeface 7    |            1 |            7
              8 | Typeface 8    |            1 |            9
              9 | Typeface 9    |            1 |            5
             10 | Typeface 10   |            1 |            2
(10 rows)
```

Рис. 2: Результат выполнения запроса 1

2.1.4 Запрос с условием

Текст запроса: Найти число начертаний, у которых количество поддерживаемых форматов больше A.
В запросе значение A принято равным 2. Сначала условие `format_count > 2` оставляет подходящие начертания, затем `COUNT` подсчитывает их. Запрос приведён в листинге 3.

Листинг 3: Подсчёт начертаний с числом форматов больше A

```
SELECT
        COUNT(v.id_font_family)
FROM typeface_format_weight_stats_view AS v
WHERE v.format_count > 2;
```

Запрос описывается формулой

$$R_2 = \gamma_{\text{count}(id_font_family)\to typeface_count}\left(\sigma_{format_count>A}(V)\right).$$

План и результат выполнения показаны на рисунках 3 и 4.

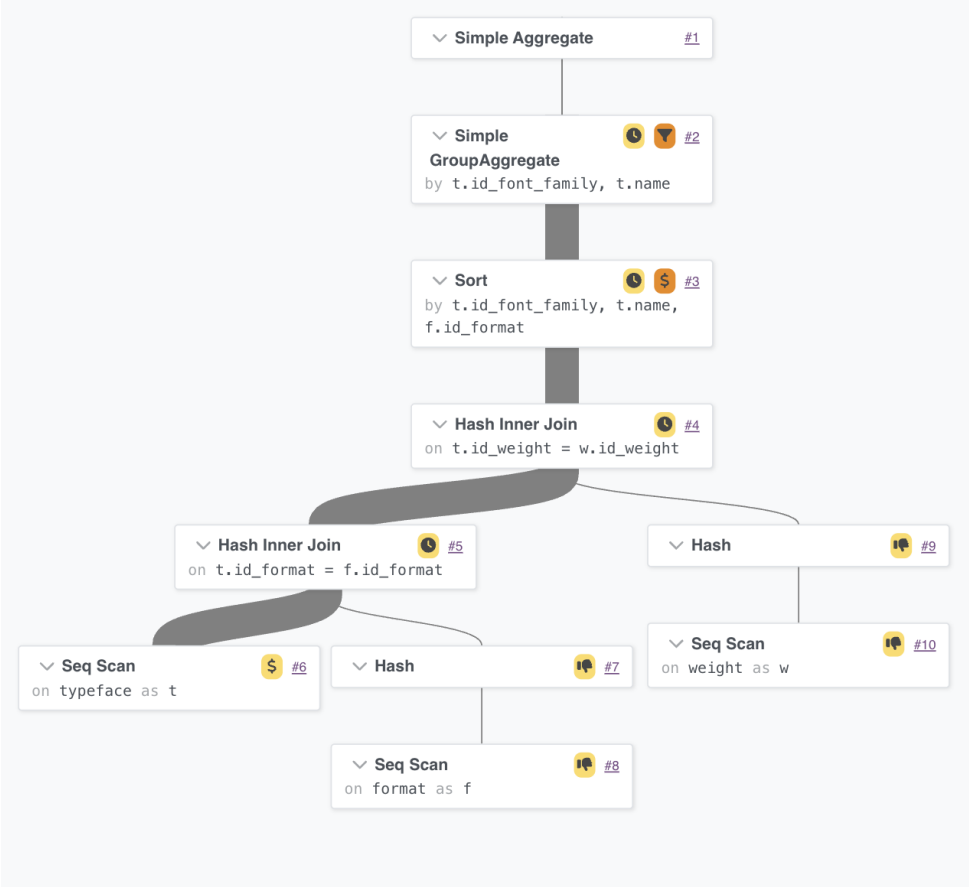


Рис. 3: План выполнения запроса 2

Сначала PostgreSQL соединяет таблицу начертаний с таблицами форматов и насыщенностей, после чего вычисляет строки представления с

помощью сортировки и `GroupAggregate`. Во время агрегирования отбрасываются начертания, у которых число форматов не больше A. Затем узел `Aggregate` подсчитывает оставшиеся строки.



Рис. 4: Результат выполнения запроса 2

Условию соответствует 39 начертаний.

2.1.5 Охват форматов по категориям

Текст запроса: Найти пять категорий с наибольшим суммарным охватом поддерживаемых форматов.
Представление содержит идентификатор гарнитуры, поэтому его можно соединить с таблицами `Font_family` и `Category`. После соединения значения `format_count` суммируются отдельно для каждой категории. Запрос приведён в листинге 4.

Листинг 4: Первые пять категорий по охвату поддерживаемых форматов

```
SELECT
    c.id_category,
    c.name,
    SUM(v.format_count) AS format_count
FROM typeface_format_weight_stats_view AS v
JOIN font_family AS ff
    ON ff.id_font_family = v.id_font_family
JOIN category AS c
    ON c.id_category = ff.id_category
GROUP BY c.id_category, c.name
ORDER BY format_count DESC, c.name
LIMIT 5;
```

Агрегирование по категориям и выбор первых пяти строк описываются формулой

$$R_3 = \operatorname{top}_5^{format_count\downarrow,\, name\uparrow}\Bigg(\gamma_{\substack{id_category,name;\\ \operatorname{sum}(format_count)\to format_count}} \big((V \bowtie_{id_font_family} FF) \bowtie_{id_category} C\big)\Bigg).$$

Представление уже содержит число форматов каждого начертания, поэтому повторно считать их в запросе не требуется. План и результат выполнения показаны на рисунках 5 и 6.

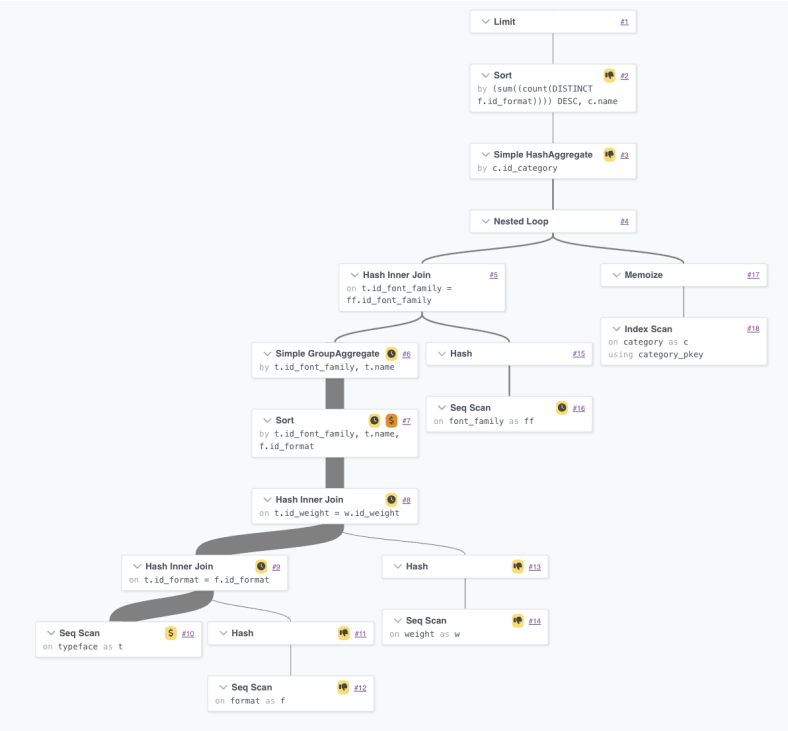


Рис. 5: План выполнения запроса 3

PostgreSQL вычисляет представление, соединяя начертания с форматами и насыщенностями. Полученные строки дополнительно соединяются с гарнитурами и категориями. Затем значения `format_count` суммируются по категориям. Завершающие сортировка и ограничение оставляют пять категорий с наибольшим охватом.

```
cw-sem4 — zsh    ✓ PostgreSQL: run current f…
/dev/fd/12:25: command not found: compdef
 id_category |     name      | format_count
-------------+---------------+--------------
          10 | Neo-grotesque |           56
           9 | Grotesque     |           54
           2 | Sans-serif    |           38
          11 | Transitional  |           33
           3 | Monospace     |           23
(5 rows)
```

Рис. 6: Результат выполнения запроса 3

После группировки категории сортируются по убыванию охвата. Оператор `LIMIT 5` оставляет первые пять строк. Наибольший охват в полученном результате имеет категория `Neo-grotesque`.

2.2 Лабораторная работа № 2

2.2.1 Задача

Необходимо реализовать таблицу `Typeface_format_weight_stats`, содержимое которой повторяет результат представления, созданного в первой лабораторной работе. Таблица должна хранить для каждого начертания идентификатор гарнитуры, название начертания, число поддерживаемых форматов и количество градаций насыщенности.
Актуальность агрегированных данных необходимо обеспечить с помощью триггеров на таблицах `Font_family` и `Typeface`. При изменении исходных данных соответствующие строки таблицы статистики должны обновляться автоматически.
Требуется реализовать пять триггеров, покрывающих следующие изменения:

1. изменение идентификатора гарнитуры в таблице `Font_family`;
2. удаление гарнитуры из таблицы `Font_family`;
3. добавление начертания в таблицу `Typeface`;
4. изменение гарнитуры, названия, формата или насыщенности начертания в таблице `Typeface`;
5. удаление начертания из таблицы `Typeface`.

Обновлённая схема базы данных представлена на рисунке 12.

2.2.2 Реализация триггеров

В PostgreSQL обработчик триггера оформляется в виде функции, возвращающей значение типа `TRIGGER`. После объявления функции команда `CREATE TRIGGER` связывает её с таблицей, событием и моментом выполнения.
Триггер `font_family_updated`**.** Триггер выполняется перед изменением идентификатора гарнитуры. Он переносит на новый идентификатор строки таблицы статистики и связанные начертания. Внешний ключ `typeface.id_font_family` объявляется отложенным, поэтому его проверка производится после завершения изменения гарнитуры. Реализация триггера приведена в листинге 5.

Листинг 5: Триггер изменения гарнитуры

```
1 ALTER TABLE typeface
2 ALTER CONSTRAINT typeface_id_font_family_fkey
```

```
DEFERRABLE INITIALLY DEFERRED;

-- обновляем таблицы дальше по цепочке
CREATE OR REPLACE FUNCTION font_family_updated_fn()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    UPDATE typeface_format_weight_stats
    SET id_font_family = NEW.id_font_family
    WHERE id_font_family = OLD.id_font_family;

    UPDATE typeface
    SET id_font_family = NEW.id_font_family
    WHERE id_font_family = OLD.id_font_family;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS font_family_updated ON font_family;

CREATE TRIGGER font_family_updated
BEFORE UPDATE OF id_font_family ON font_family
FOR EACH ROW
EXECUTE FUNCTION font_family_updated_fn();
```

Триггер `font_family_deleted`**.** Триггер выполняется перед удалением гарнитуры. Сначала он удаляет принадлежащие ей начертания, после чего удаляет соответствующие агрегированные строки. Это позволяет выполнить операцию без каскадного удаления на уровне внешнего ключа. Реализация приведена в листинге 6.

Листинг 6: Триггер удаления гарнитуры

```
-- удаляем всё дальше по цепочке
CREATE OR REPLACE FUNCTION font_family_deleted_fn()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    DELETE FROM typeface
    WHERE id_font_family = OLD.id_font_family;

    DELETE FROM typeface_format_weight_stats
    WHERE id_font_family = OLD.id_font_family;

    RETURN OLD;
END;
$$;

```

```
DROP TRIGGER IF EXISTS font_family_deleted ON font_family;

CREATE TRIGGER font_family_deleted
BEFORE DELETE ON font_family
FOR EACH ROW
EXECUTE FUNCTION font_family_deleted_fn();
```

Триггер `typeface_inserted`**.** После добавления начертания триггер подсчитывает число различных форматов и градаций насыщенности для соответствующей пары из идентификатора гарнитуры и названия начертания. Если агрегированная строка уже существует, её значения обновляются конструкцией `ON CONFLICT`. Реализация приведена в листинге 7.

Листинг 7: Триггер добавления начертания

```
CREATE OR REPLACE FUNCTION typeface_inserted_fn()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO typeface_format_weight_stats
    SELECT
        t.id_font_family,
        t.name,
        COUNT(DISTINCT f.id_format),
        COUNT(DISTINCT w.id_weight)
    FROM typeface AS t
    JOIN format AS f ON f.id_format = t.id_format
    JOIN weight AS w ON w.id_weight = t.id_weight
    WHERE t.id_font_family = NEW.id_font_family
      AND t.name = NEW.name
    GROUP BY
        t.id_font_family,
        t.name
    ON CONFLICT (id_font_family, typeface_name)
    DO UPDATE SET
        format_count = EXCLUDED.format_count,
        weight_count = EXCLUDED.weight_count;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS typeface_inserted ON typeface;

CREATE TRIGGER typeface_inserted
AFTER INSERT ON typeface
FOR EACH ROW
EXECUTE FUNCTION typeface_inserted_fn();
```

Триггер `typeface_updated`**.** Триггер запускается после изменения гарни-

туры, названия, формата или насыщенности начертания. Старая агрегированная строка удаляется, затем по актуальным данным заново рассчитываются показатели старой и новой групп. Реализация приведена в листинге 8.

Листинг 8: Триггер изменения начертания

```
CREATE OR REPLACE FUNCTION typeface_updated_fn()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    DELETE FROM typeface_format_weight_stats
    WHERE id_font_family = OLD.id_font_family
      AND typeface_name = OLD.name;

    INSERT INTO typeface_format_weight_stats
    SELECT
        t.id_font_family,
        t.name,
        COUNT(DISTINCT f.id_format),
        COUNT(DISTINCT w.id_weight)
    FROM typeface AS t
    JOIN format AS f ON f.id_format = t.id_format
    JOIN weight AS w ON w.id_weight = t.id_weight
    WHERE (t.id_font_family = OLD.id_font_family AND t.name = OLD.name)
       OR (t.id_font_family = NEW.id_font_family AND t.name = NEW.name)
    GROUP BY
        t.id_font_family,
        t.name
    -- чтобы избежать ошибки дублирования первичного ключа
    ON CONFLICT (id_font_family, typeface_name)
    DO UPDATE SET
        format_count = EXCLUDED.format_count,
        weight_count = EXCLUDED.weight_count;

    RETURN NEW;
END;
$$;

DROP TRIGGER IF EXISTS typeface_updated ON typeface;

CREATE TRIGGER typeface_updated
AFTER UPDATE OF id_font_family, name, id_format, id_weight ON typeface
FOR EACH ROW
EXECUTE FUNCTION typeface_updated_fn();
```

Триггер `typeface_deleted`**.** После удаления начертания триггер удаляет прежнюю агрегированную строку и повторно рассчитывает её по оставшимся начертаниям. Если записей с такой парой гарнитуры и названия больше нет, новая строка не создаётся. Реализация приведена в листин-

ге 9.

Листинг 9: Триггер удаления начертания

```
CREATE OR REPLACE FUNCTION typeface_deleted_fn()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    -- удаляем начертание и обновляем значение в табличке
    DELETE FROM typeface_format_weight_stats
    WHERE id_font_family = OLD.id_font_family
      AND typeface_name = OLD.name;

    INSERT INTO typeface_format_weight_stats
    SELECT
        t.id_font_family,
        t.name,
        COUNT(DISTINCT f.id_format),
        COUNT(DISTINCT w.id_weight)
    FROM typeface AS t
    JOIN format AS f ON f.id_format = t.id_format
    JOIN weight AS w ON w.id_weight = t.id_weight
    WHERE t.id_font_family = OLD.id_font_family
      AND t.name = OLD.name
    GROUP BY
        t.id_font_family,
        t.name;

    RETURN NULL;
END;
$$;

DROP TRIGGER IF EXISTS typeface_deleted ON typeface;

CREATE TRIGGER typeface_deleted
AFTER DELETE ON typeface
FOR EACH ROW
EXECUTE FUNCTION typeface_deleted_fn();
```

2.2.3 Проверка работы триггеров

Для каждого триггера подготовлен отдельный тестовый запрос. Все проверки выполняются внутри транзакции, которая завершается командой ROLLBACK; поэтому тестовые изменения не сохраняются в базе данных. Рассмотренные сценарии приведены в таблице 1.

Таблица 1: Примеры работы триггеров

№	Действие
1	Изменение идентификатора гарнитуры (`UPDATE id_font_family` в `Font_family`). Идентификатор изменяется в связанных начертаниях и строках `Typeface_format_weight_stats`.
2	Удаление гарнитуры (`DELETE` из `Font_family`). Связанные начертания и агрегированные строки удаляются триггером.
3	Добавление начертания (`INSERT` в `Typeface`). В таблице статистики создаётся новая строка либо пересчитываются существующие счётчики.
4	Изменение начертания (`UPDATE` полей `id_font_family`, `name`, `id_format` или `id_weight`). Показатели старой и новой групп пересчитываются.
5	Удаление начертания (`DELETE` из `Typeface`). Счётчики группы пересчитываются, а при отсутствии других начертаний агрегированная строка удаляется.

Результаты выполнения тестовых запросов представлены на рисунках 7–11.

```
UPDATE 1
 id_font_family | typeface_name | format_count | weight_count
----------------+---------------+--------------+--------------
          10000 | Typeface 1    |            1 |            5
(1 row)
```

Рис. 7: Результат изменения идентификатора гарнитуры

```
INSERT 0 1
INSERT 0 1
DELETE 1
 count
-------
     0
(1 row)
```

Рис. 8: Результат удаления гарнитуры

```
INSERT 0 1
 id_font_family |    typeface_name    | format_count | weight_count
----------------+---------------------+--------------+--------------
              1 | Trigger insert test |            1 |            1
(1 row)
```

Рис. 9: Результат добавления начертания

```
UPDATE 1
 id_font_family |    typeface_name    | format_count | weight_count
----------------+---------------------+--------------+--------------
              1 | Trigger update test |            1 |            1
(1 row)
```

Рис. 10: Результат изменения начертания

```
INSERT 0 1
DELETE 1
 count
-------
     0
```

Рис. 11: Результат удаления начертания

2.2.4 Схема базы данных

В исходную схему добавлена таблица `Typeface_format_weight_stats`. Пунктирные стрелки показывают операции, при выполнении которых триггеры изменяют содержащиеся в ней агрегированные значения. Обновлённая схема представлена на рисунке 12.

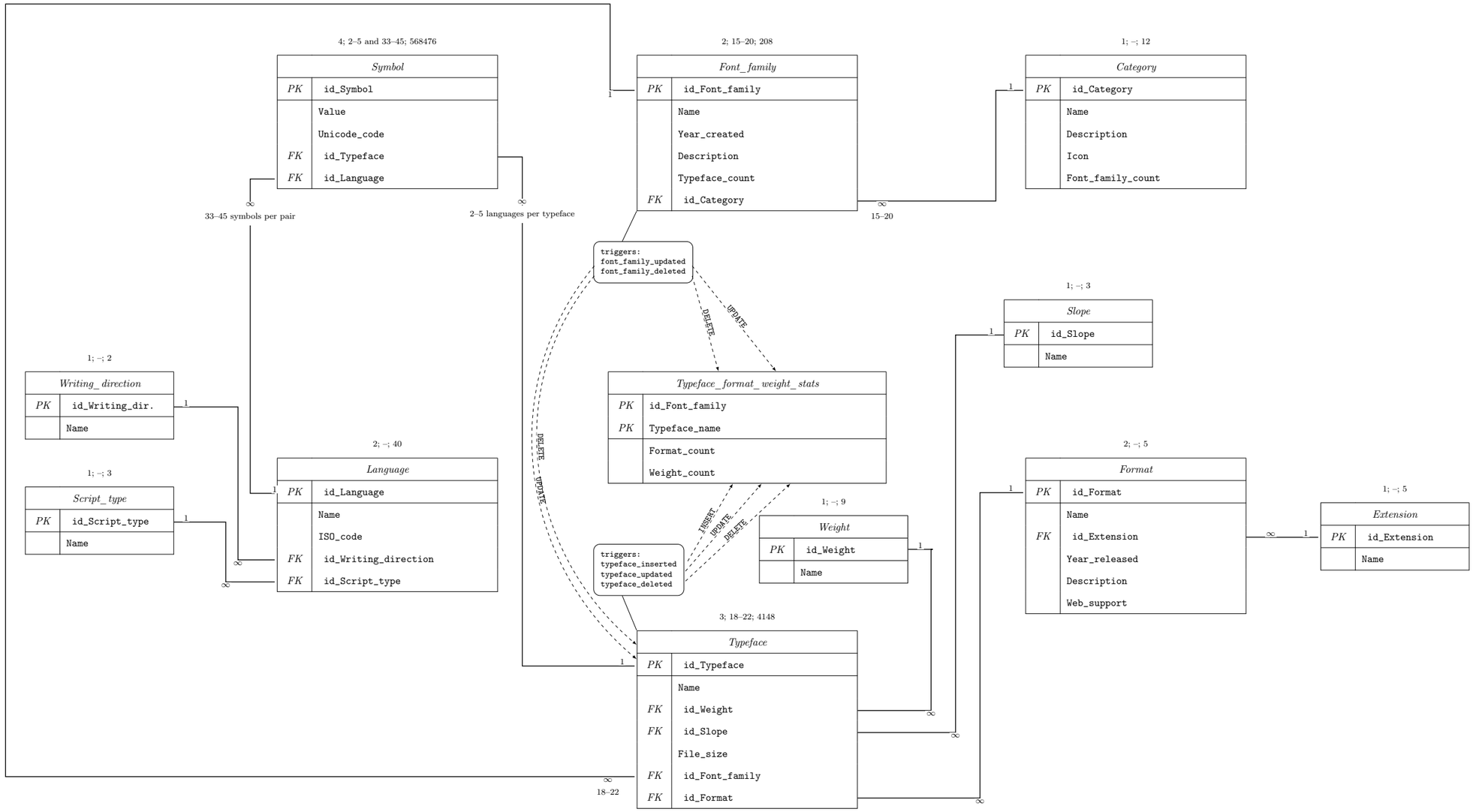


Рис. 12: Схема базы данных с таблицей статистики и триггерами

2.3 Лабораторная работа № 3

2.3.1 Задача

Создать двух пользователей базы данных, обладающих различными правами доступа. Первый пользователь должен быть предназначен для просмотра статистики по начертаниям. Второй пользователь должен иметь возможность работать с агрегированными и исходными данными, связанными с этой статистикой.

1. Пользователь `stats_reader` может только просматривать таблицу `typeface_format_weight_stats`. Ему доступна операция `SELECT` из этой таблицы, ко всем остальным таблицам доступа нет.

2. Пользователь `source_editor` имеет полный доступ к таблицам `Font_family` и `Typeface`, изменения которых влияют на таблицу статистики. Ему также разрешено чтение и изменение `typeface_format_weight_stats`. Доступ к остальным таблицам базы данных запрещён.

Для проверки назначенных прав требуется выполнить один набор тестовых запросов от имени обоих пользователей и сопоставить полученные результаты.

2.3.2 Создание пользователей и назначение прав

Командами `CREATE USER` создаются пользователи `stats_reader` и `source_editor`. Обоим пользователям разрешаются подключение к базе данных, использование схемы `public` и чтение таблицы статистики. Пользователь `source_editor` дополнительно получает права на изменение связанных таблиц и используемые ими последовательности. Полный скрипт приведён в листинге 10.

Листинг 10: Создание пользователей и назначение прав

```
1  DROP OWNED BY stats_reader;
2  DROP OWNED BY source_editor;
3
4  DROP USER IF EXISTS stats_reader;
5  DROP USER IF EXISTS source_editor;
6
7  CREATE USER stats_reader PASSWORD 'stats_reader';
8  CREATE USER source_editor PASSWORD 'source_editor';
9
10 GRANT CONNECT ON DATABASE typography
11 TO stats_reader, source_editor;
12
```

```
13 GRANT USAGE ON SCHEMA public
14 TO stats_reader, source_editor;
15
16 GRANT SELECT ON typeface_format_weight_stats
17 TO stats_reader, source_editor;
18
19 GRANT SELECT, INSERT, UPDATE, DELETE ON font_family, typeface
20 TO source_editor;
21
22 GRANT USAGE, SELECT ON SEQUENCE
23     font_family_id_font_family_seq,
24     typeface_id_typeface_seq
25 TO source_editor;
26
27 GRANT INSERT, UPDATE, DELETE
28 ON TABLE font_family, typeface, typeface_format_weight_stats
29 TO source_editor;
```

2.3.3 Тестовые примеры

Для проверки прав подготовлено десять запросов. Каждый запрос выполняется без изменений сначала от имени `stats_reader`, затем от имени `source_editor`.
Первый запрос проверяет чтение таблицы статистики. Эта операция должна быть доступна обоим пользователям. Текст запроса приведён в листинге 11.

Листинг 11: Чтение таблицы статистики

```
1 SELECT * FROM typeface_format_weight_stats LIMIT 3;
```

Второй запрос проверяет возможность непосредственного добавления строки таблицы статистики. Третий запрос читает её по составному первичному ключу, а четвёртый удаляет. Тексты запросов приведены в листингах 12–14.

Листинг 12: Добавление строки таблицы статистики

```
1 INSERT INTO typeface_format_weight_stats (
2     id_font_family,
3     typeface_name,
4     format_count,
5     weight_count
6 )
7 VALUES (-1, 'Permission test', 1, 1);
```

Листинг 13: Проверка добавленной строки таблицы статистики

```sql
SELECT *
FROM typeface_format_weight_stats
WHERE id_font_family = -1
  AND typeface_name = 'Permission test';
```

Листинг 14: Удаление строки таблицы статистики

```sql
DELETE FROM typeface_format_weight_stats
WHERE id_font_family = -1;
```

Пятый запрос добавляет тестовое начертание, копируя значения внешних ключей из первой существующей строки. Текст запроса приведён в листинге 15.

Листинг 15: Добавление тестового начертания

```sql
INSERT INTO typeface (
    id_typeface,
    name,
    id_weight,
    id_slope,
    file_size,
    id_font_family,
    id_format
)
SELECT
    1000000,
    'Permission test',
    id_weight,
    id_slope,
    file_size,
    id_font_family,
    id_format
FROM typeface
ORDER BY id_typeface
LIMIT 1;
```

Шестой запрос проверяет наличие добавленного начертания в исходной таблице `Typeface`. Седьмой запрос изменяет его название, а восьмой выбирает изменённую строку по идентификатору. Тексты запросов приведены в листингах 16–18.

Листинг 16: Проверка добавленного начертания

```sql
SELECT id_typeface, name
FROM typeface
WHERE name = 'Permission test';
```

Листинг 17: Изменение тестового начертания

```sql
UPDATE typeface
SET name = 'Permission test updated'
WHERE id_typeface = 1000000;
```

Листинг 18: Проверка изменённого начертания

```sql
SELECT id_typeface, name
FROM typeface
WHERE id_typeface = 1000000;
```

Девятый запрос удаляет добавленное ранее тестовое начертание. Десятый запрос обращается к таблице `Category`, права на которую не были назначены ни одному из пользователей. Тексты запросов приведены в листингах 19 и 20.

Листинг 19: Удаление тестового начертания

```sql
DELETE FROM typeface
WHERE id_typeface = 1000000;
```

Листинг 20: Чтение таблицы без назначенных прав

```sql
SELECT *
FROM category
LIMIT 3;
```

2.3.4 Результаты тестирования

Результаты выполнения одного набора запросов от имени двух пользователей сопоставлены в таблице 2. В первой строке каждого примера приведён текст запроса, во второй — результаты его выполнения.

Таблица 2: Результаты тестовых запросов

№	stats_reader	source_editor
1	`SELECT * FROM typeface_format_weight_stats LIMIT 3;`	

	`typography=> SELECT * FROM typeface_format_weight_stats LIMIT 3;`
	` id_font_family | typeface_name | format_count | weight_count`
	`---------------+---------------+--------------+--------------`
	`1 | Typeface 1 | 1 | 5`
	`2 | Typeface 2 | 1 | 2`
	`3 | Typeface 3 | 1 | 9`
	`(3 rows)`
		`typography=> SELECT * FROM typeface_format_weight_stats LIMIT 3;`
		` id_font_family | typeface_name | format_count | weight_count`
		`---------------+---------------+--------------+--------------`
		`1 | Typeface 1 | 1 | 5`
		`2 | Typeface 2 | 1 | 2`
		`3 | Typeface 3 | 1 | 9`
		`(3 rows)`

2	`INSERT INTO typeface_format_weight_stats (...) VALUES (-1, 'Permission test', 1, 1);`	

	`typography=> INSERT INTO typeface_format_weight_stats (`
	`id_font_family,`
	`typeface_name,`
	`format_count,`
	`weight_count`
	`)`
	`VALUES (-1, 'Permission test', 1, 1);`
	`ERROR: permission denied for table typeface_format_weight_stats`
	`typography=>`
		`typography=> INSERT INTO typeface_format_weight_stats (`
		`id_font_family,`
		`typeface_name,`
		`format_count,`
		`weight_count`
		`)`
		`VALUES (-1, 'Permission test', 1, 1);`
		`INSERT 0 1`
		`typography=>`

3	`SELECT * FROM typeface_format_weight_stats WHERE id_font_family = -1 AND typeface_name = 'Permission test';`	

	`FROM typeface_format_weight_stats`
	`WHERE id_font_family = -1`
	`AND typeface_name = 'Permission test';`
	` id_font_family | typeface_name | format_count | weight_count`
	`---------------+-----------------+--------------+--------------`
	`-1 | Permission test | 1 | 1`
	`(1 row)`
		`FROM typeface_format_weight_stats`
		`WHERE id_font_family = -1`
		`AND typeface_name = 'Permission test';`
		` id_font_family | typeface_name | format_count | weight_count`
		`---------------+-----------------+--------------+--------------`
		`-1 | Permission test | 1 | 1`
		`(1 row)`

4	`DELETE FROM typeface_format_weight_stats WHERE id_font_family = -1;`	

	`typography=> DELETE FROM typeface_format_weight_stats`
	`WHERE id_font_family = -1;`
	`ERROR: permission denied for table typeface_format_weight_stats`
		`typography=> DELETE FROM typeface_format_weight_stats`
		`WHERE id_font_family = -1;`
		`DELETE 1`

5	`INSERT INTO typeface (...) SELECT ... FROM typeface ORDER BY id_typeface LIMIT 1;`	

	`typography=> INSERT INTO typeface (`
	`id_typeface,`
	`name,`
	`id_weight,`
	`id_slope,`
	`file_size,`
	`id_font_family,`
	`id_format`
	`)`
	`SELECT`
	`1000000,`
	`'Permission test',`
	`id_weight,`
	`id_slope,`
	`file_size,`
	`id_font_family,`
	`id_format`
	`FROM typeface`
	`ORDER BY id_typeface`
	`LIMIT 1;`
	`ERROR: permission denied for table typeface`
		`typography=> INSERT INTO typeface (`
		`id_typeface,`
		`name,`
		`id_weight,`
		`id_slope,`
		`file_size,`
		`id_font_family,`
		`id_format`
		`)`
		`SELECT`
		`1000000,`
		`'Permission test',`
		`id_weight,`
		`id_slope,`
		`file_size,`
		`id_font_family,`
		`id_format`
		`FROM typeface`
		`ORDER BY id_typeface`
		`LIMIT 1;`
		`INSERT 0 1`

6	`SELECT id_typeface, name FROM typeface WHERE name = 'Permission test';`	

	`typography=> SELECT id_typeface, name`
	`FROM typeface`
	`WHERE name = 'Permission test';`
	`ERROR: permission denied for table typeface`
	`typography=>`
		`typography=> SELECT id_typeface, name`
		`FROM typeface`
		`WHERE name = 'Permission test';`
		` id_typeface | name`
		`-------------+-----------------`
		`1000000 | Permission test`
		`(1 row)`

7	`UPDATE typeface SET name = 'Permission test updated' WHERE id_typeface = 1000000;`	

	`typography=> UPDATE typeface`
	`SET name = 'Permission test updated'`
	`WHERE id_typeface = 1000000;`
	`ERROR: permission denied for table typeface`
		`typography=> UPDATE typeface`
		`SET name = 'Permission test updated'`
		`WHERE id_typeface = 1000000;`
		`UPDATE 1`

8	`SELECT id_typeface, name FROM typeface WHERE id_typeface = 1000000;`	

	`typography=> SELECT id_typeface, name`
	`FROM typeface`
	`WHERE id_typeface = 1000000;`
	`ERROR: permission denied for table typeface`
	`typography=>`
		`typography=> SELECT id_typeface, name`
		`FROM typeface`
		`WHERE id_typeface = 1000000;`
		` id_typeface | name`
		`-------------+-------------------------`
		`1000000 | Permission test updated`
		`(1 row)`

9	`DELETE FROM typeface WHERE id_typeface = 1000000;`	

	`typography=> DELETE FROM typeface`
	`WHERE id_typeface = 1000000;`
	`ERROR: permission denied for table typeface`
		`typography=> DELETE FROM typeface`
		`WHERE id_typeface = 1000000;`
		`DELETE 1`

10	`SELECT * FROM category LIMIT 3;`	

	`typography=> SELECT *`
	`FROM category`
	`LIMIT 3;`
	`ERROR: permission denied for table category`
		`typography=> SELECT *`
		`FROM category`
		`LIMIT 3;`
		`ERROR: permission denied for table category`

2.4 Лабораторная работа № 4

2.4.1 Задача

Выбрать уровень изоляции и аномалию параллельного выполнения транзакций. Сформировать сценарий из двух транзакций A и B, подтверждающий наличие или отсутствие аномалии в PostgreSQL, выполнить его в двух сеансах `psql` и обосновать результат.

2.4.2 Сценарий эксперимента

Выбраны уровень `READ UNCOMMITTED` и фантомное чтение. В PostgreSQL данный уровень работает как `READ COMMITTED`, а фантом проявляется, если повторный запрос с тем же условием видит новую зафиксированную строку.
Транзакция A подсчитывает строки таблицы `Font_family`, название которых начинается с `Phantom read test`. Транзакция B добавляет подходящую строку и фиксирует изменение, после чего A повторяет запрос. Команды приведены в листингах 21 и 22.

Листинг 21: Сценарий транзакции A

```
DELETE FROM font_family
WHERE name = 'Phantom read test family';

BEGIN;
SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;

-- до вставки в сеансе B ожидается 0 строк
SELECT COUNT(*) AS matching_families
FROM font_family
WHERE name LIKE 'Phantom read test%';

-- выполняем INSERT в сеансе B

-- проверка на грязное чтение, должно быть 0 строк (т.к. READ UNCOMMITTED
↪  эквивалентен READ COMMITTED)
SELECT COUNT(*) AS matching_families
FROM font_family
WHERE name LIKE 'Phantom read test%';

-- выполняем COMMIT в сеансе B

-- после COMMIT возникает фантом: ожидается 1 строка
SELECT COUNT(*) AS matching_families
FROM font_family
WHERE name LIKE 'Phantom read test%';

COMMIT;
```

Листинг 22: Сценарий транзакции B

```
BEGIN;

INSERT INTO font_family (
    name,
    typeface_count,
    id_category
)
SELECT
    'Phantom read test family',
    0,
    id_category
FROM font_family
ORDER BY id_font_family
LIMIT 1
RETURNING id_font_family, name;

-- выполняем второе чтение в сеансе A

COMMIT;
```

Хронология выполнения команд представлена в таблице 3; прочерк обозначает отсутствие действия в соответствующем сеансе.

Таблица 3: Выполнение сценария фантомного чтения на уровне `READ UNCOMMITTED`

№	Действия транзакции A	Действия транзакции B

1	`typography=# BEGIN;`
	`SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED;`
	`BEGIN`
	`SET`
		—

2	`typography=*# SELECT COUNT(*) AS matching_families`
	`FROM font_family`
	`WHERE name LIKE 'Phantom read test%';`
	` matching_families`
	`------------------`
	`0`
	`(1 row)`
		—

3	—	`typography=# BEGIN;` `BEGIN`

4	—	`typography=*# INSERT INTO font_family (`
		`name,`
		`typeface_count,`
		`id_category`
		`)`
		`SELECT`
		`'Phantom read test family',`
		`0,`
		`id_category`
		`FROM font_family`
		`ORDER BY id_font_family`
		`LIMIT 1`
		`RETURNING id_font_family, name;`
		` id_font_family | name`
		`----------------+--------------------------`
		`210 | Phantom read test family`
		`(1 row)`
		
		`INSERT 0 1`

5	`typography=*# SELECT COUNT(*) AS matching_families`
	`FROM font_family`
	`WHERE name LIKE 'Phantom read test%';`
	` matching_families`
	`------------------`
	`0`
	`(1 row)`
		—

6	—	`typography=*# COMMIT;` `COMMIT`

7	`typography=*# SELECT COUNT(*) AS matching_families`
	`FROM font_family`
	`WHERE name LIKE 'Phantom read test%';`
	` matching_families`
	`------------------`
	`1`
	`(1 row)`
		—

8	`typography=*# COMMIT;` `COMMIT`	—

2.4.3 Результат

Результаты чтений в транзакции A равны $0 \to 0 \to 1$. Незавершённая вставка B не видна, поэтому грязное чтение отсутствует. После фиксации B тот же запрос видит новую строку, следовательно, наблюдается фантомное чтение. Это соответствует поведению `READ COMMITTED`: каждый новый запрос видит изменения, которые другие транзакции уже зафиксировали.

3 Заключение